

DEAD BY DAYLIGHT TURN-BASED RPG

Documentación Técnica de Ingeniería de Software y Base de Datos Relacional

1. Introducción General del Proyecto

El proyecto 'Dead by Daylight RPG' es una adaptación interactiva y por turnos basada en el popular juego de terror asimétrico. Desarrollada sobre una arquitectura de software full-stack moderna, la aplicación combina un Backend robusto escrito en Java (Spring Boot / Maven / MariaDB) y un Frontend de alta fidelidad estética estructurado en Next.js (TypeScript, Tailwind CSS, Framer Motion, WebSockets).

Este sistema implementa mecánicas avanzadas como autenticación segura basada en tokens (JWT), persistencia robusta de estados de juego con eliminación en cascada, lógica de combate automatizada por IA o control manual directo, guardados automáticos (Autosave) sincronizados y un sistema dinámico de escalado de dificultad personalizado según el bando seleccionado (Supervivientes o Asesinos).

2. Arquitectura de Base de Datos y Modelo de Persistencia

El modelo de persistencia del proyecto está diseñado bajo una base de datos relacional MariaDB. Garantiza consistencia transaccional e integridad referencial, asegurando que el complejo estado de una partida en curso pueda representarse, guardarse y restaurarse fielmente sin corromper el flujo de juego.

2.1 Catálogo de Tablas del Sistema

Tabla	Descripción	Columnas y Claves
USUARIO	Almacena las credenciales y perfiles de los jugadores.	id (INT PK AutoInc), username (VARCHAR Unique), password (VARCHAR)
RANURA	Representa cada slot de guardado por usuario (1-3) y su estado global de ronda.	id_ranura (INT PK), usuario_id (INT PK FK), modo_juego, ronda, terminada, bando, dificultad
PERSONAJE_PARTIDA	Contiene las entidades instanciadas (Leon, Ghostface, etc.) en cada slot.	id (INT PK AutoInc), id_ranura (INT FK), usuario_id (INT FK), nombre, vida_max, vida_actual, defensa_base, es_killer
ARMA_EQUIPADA	Representa el arma asignada a un personaje y su potencia de ataque.	id (INT PK AutoInc), personaje_id (INT FK Unique), nombre, danio_base
PERK_EQUIPADA	Efectos y perks activos con su contador de usos.	id (INT PK AutoInc), personaje_id (INT FK), nombre, usos_restantes
ESTADO_ACTIVO	Gestiona la posición y estados activos en combate por personaje.	id (INT PK AutoInc), personaje_id (INT FK Unique), turno_activo, defendiendo
LOGRO	Catálogo maestro estático de logros de juego.	id (INT PK AutoInc), nombre (VARCHAR Unique), descripcion
LOGRO_CONSEGUIDO	Registra los logros desbloqueados por cada usuario.	id (INT PK AutoInc), usuario_id (INT FK), logro_id (INT FK), fecha_conseguido

2.2 Explicación del Diagrama EER (Entidad-Relación Extendido)

El diagrama EER (Enhanced Entity-Relationship) refleja una jerarquía lógica altamente estructurada:

- **Relación 1-a-N entre USUARIO y RANURA:** Un usuario posee hasta 3 ranuras asociadas de forma unívoca a través de una clave compuesta (id_ranura, usuario_id).
- **Relación 1-a-N entre RANURA y PERSONAJE_PARTIDA:** Cada ranura de guardado activa contiene exactamente 6 personajes cargados en combate (3 supervivientes y 3 asesinos). Al borrar la ranura, las entidades asociadas se eliminan en cascada gracias a las reglas ON DELETE CASCADE.
- **Relación 1-a-1 entre PERSONAJE_PARTIDA y ARMA_EQUIPADA / ESTADO_ACTIVO:** Cada personaje posee un único perfil de arma y un único registro de combate activo.
- **Relación 1-a-N entre PERSONAJE_PARTIDA y PERK_EQUIPADA:** Un personaje puede equipar múltiples habilidades que restan usos en combate.
- **Relación N-a-M entre USUARIO y LOGRO:** Registrada mediante la tabla asociativa LOGRO_CONSEGUIDO para almacenar las hazañas del jugador.

3. Programación, Arquitectura de Software y Lógica Aplicada

3.1 Seguridad y el Mecanismo de JWT (JSON Web Tokens)

El backend implementa autenticación sin estado (Stateless) usando **Spring Security** y **JWT (JSON Web Tokens)**. Esto permite verificar la autenticidad del cliente en cada endpoint REST sin depender de sesiones locales del servidor.

JwtUtil.java (Servicio de Criptografía de Tokens): Encargado de la lógica criptográfica de bajo nivel. Firma los tokens utilizando algoritmos criptográficos (HMAC-SHA256) con una clave secreta segura, extrae declaraciones (Claims) como el nombre de usuario y comprueba la validez y expiración de los tokens recibidos.

JwtFilter.java (Filtro interceptor HTTP): Este componente intercepta todas las peticiones entrantes. Si detecta la cabecera 'Authorization' con el formato 'Bearer <token>', valida el token a través de JwtUtil. Si es correcto, inyecta las credenciales del usuario al contexto de Spring Security (SecurityContextHolder), permitiendo autorizar el acceso y asociar operaciones al perfil del usuario.

3.2 Lógica de Negocio y Capa de Servicio (Service Layer)

La capa lógica cuenta con dos núcleos principales:

- **TrialService.java (Orquestador principal):** Interconecta los endpoints REST con la base de datos a través de GestorPersistencia.java. Se encarga de instanciar personajes de forma dinámica usando **Java Reflection** según el nombre de la entidad en base de datos y de inicializar, reanudar o guardar partidas.
- **MotorTrial.java (El motor de turnos):** Administra el bucle de turnos, el cálculo de daño de combate, la reducción de usos de Perks, los modificadores de defensa y la declaración de finalización del combate.

3.3 Guardado Automático (Autosave) y Escalado de Dificultad

Guardado Automático por Ronda: Cuando inicias una Nueva Partida, seleccionas un slot por defecto (1-3). Al concluir cada ronda en el combate (de asesinos de vuelta a supervivientes), el frontend Next.js realiza una llamada en segundo plano a '/api/game/save' de manera totalmente silenciosa, persistiendo de forma segura el progreso en base de datos.

Escalado de Dificultad Dinámico (applyDifficultyScaling): Modifica las estadísticas iniciales del bando opuesto basándose en el bando seleccionado y el nivel de dificultad:

- **Fácil:** Reduce en un 30% (factor de escala 0.7) los atributos de HP, defensa y daño de armas del bando rival.
- **Normal:** Estadísticas base al 100%.
- **Extremo:** Aumenta en un 40% (factor 1.4) el HP máximo y la defensa de los enemigos, y en un 30% (factor 1.3) el daño de sus ataques.

Documento de especificación técnica oficial generado exitosamente para propósitos académicos del curso.